

TLA-Prover Addendum: The Verifier as Controller, and a Decode-Time Grammar for TLA⁺

Eric Spencer, Arslan Bisharat, Brian Ortiz, Khushboo Bhadauria, TaiNing Wang, George K. Thiruvathukal, Konstantin Läufer, Mohammed Abuhamad

*Department of Computer Science, Loyola University Chicago, Chicago, IL 60660, USA
{espencer2, marslan, bortiz4, kbhadauria, twang12, gthiruvathukal, klaufer, mabuhamad}@luc.edu*

Keywords:

TLA⁺, Large Language Models, Formal Methods, Specification Synthesis, Constrained Decoding, Verifier-in-the-Loop.

Abstract:

TLA-Prover evaluates TLA⁺ generation open-loop: k independent draws from one prompt, each scored once by SANY and TLC, best-of reported. Every arm in this program has used that framing, including TLA-Prover’s own headline number. The verifier acts as a judge, not a controller. This addendum closes the loop: generate, verify, feed the diagnosis back, regenerate. On the frozen 30-spec holdout, an untuned base **gpt-oss-120b** run in the loop solves a mean of 17.3/30 across three seeds (18/16/18). The matched open-loop control on the same endpoint solves a mean of 10.7/30 across three seeds (12/8/12), and the two ranges do not overlap. The closed loop reaches this with fewer model calls per spec (571–616) than the open-loop control (960–990). No fine-tuning is involved; the checkpoint is the untuned base model. The best fine-tuned checkpoint in this program reaches 15/30 open-loop against a 12/30 base control, a gain the program’s own pre-registered rule marks null (McNemar $p = 0.45$). The untuned base model run in the loop, at a mean of 17.3/30, exceeds that fine-tuned checkpoint. We also catalog why generation fails: across 3,836 SANY failures pooled over six arms, 51% do not parse and 34% invoke an unknown operator. Separately, we measure a decode-time grammar for constrained decoding, `tla_module_v1.ebnf`, against 438 known-good specs and 1,543 real parse failures. It false-rejects none of the 438 valid specs and would reject 86.7% of the real failures. The grammar has not been run inside a generation arm; the 86.7% is an offline upper bound on what decode-time constraint could remove, not a measured pass-rate gain.

1 Introduction

TLA-Prover pairs TLA⁺ (Lamport, 2002) generation with the TLC model checker (Yu et al., 1999): a deterministic verifier with a binary success criterion. Every evaluation in that paper, and every arm run in the training program that followed it, uses the verifier the same way. A prompt produces k independent draws, SANY and TLC score each draw once, and the best draw is reported. The verifier judges; it never talks back to the model that produced the failure.

This is a narrower use of the verifier than the setup allows. SANY and TLC both return structured diagnostics: a line number and message on a parse failure, a counterexample trace

on a model-check failure. Nothing in the open-loop evaluation reads that diagnostic back into the next draw. This addendum does: generate, verify, feed the diagnosis back, regenerate. We call this framing L, matched against the parent paper’s open-loop framing (framing A) on prompt text and on model-call budget, so that neither arm wins by spending more.

Framing L is run on the untuned base model behind the training program, **gpt-oss-120b** (OpenAI, 2025), with no task-specific training of any kind. On the frozen 30-spec holdout, three seeds each, framing L solves a mean of 17.3/30 (18/16/18) against framing A’s mean of 10.7/30 (12/8/12). The two ranges do not overlap, and framing L uses fewer

model calls per spec (571–616) than framing A (960–990). The best fine-tuned checkpoint produced by this training program reaches 15/30 under framing A against a 12/30 base control, a result the program’s own pre-registered rule marks null. The untuned base model under framing L exceeds it. Closing the loop, on this benchmark, moves the number further than the fine-tuning program has so far.

We also report why generation fails and what a decode-time grammar can catch. Pooled over 3,836 SANY failures across six arms, 51% of failing candidates do not parse at all and 34% invoke an unknown operator. A grammar for the TLA⁺ expression language, `tla_module_v1.ebnf`, is measured offline against 438 known-good specifications and 1,543 real parse failures: zero false rejects, 86.7% of the real failures caught. A companion negative result narrows the interpretation: a deterministic lint pass that forces 88 failing candidates to parse flips zero of them to a TLC pass. Parsing is not the bottleneck once a candidate is forced past it.

Contributions. ① Framing L, a closed-loop generation procedure that feeds SANY/TLC diagnosis back into regeneration, matched to the parent paper’s open-loop framing on prompt and model-call budget, together with the measurement that it exceeds both the open-loop control and the program’s best fine-tuned checkpoint on an untuned base model. ② A failure taxonomy over 3,836 pooled SANY failures, decomposing the 34% unknown-operator class into five sub-causes and identifying the tokens most associated with parse failure. ③ An offline measurement of a decode-time grammar for constrained TLA⁺ decoding, `tla_module_v1.ebnf`, with its false-reject and catch rates, and a negative result showing that forcing output to parse does not by itself recover any additional solved spec.

Organization. Section 3 describes framing L and the grammar measurement; Section 4 reports the seed-level comparison, the failure taxonomy, and the grammar numbers; Section 5 states what this addendum does not establish; Section 6 concludes.

2 Related Work

Verifier-in-the-Loop Generation. Bal-dur (First et al., 2023) generates and repairs whole Isabelle proofs against the verifier’s own feedback. DeepSeek-Prover (Xin et al., 2024)

uses a deterministic verifier as the reward signal for large-scale synthetic proof data. Both treat the verifier as part of the generation loop rather than as a final judge. Framing L applies the same idea to TLA⁺: the verifier’s diagnostic, not just its pass/fail bit, is fed back into the next draw.

Grammar-Guided Decoding. Grammar-constrained decoding restricts a language model’s output to a formal grammar (Geng et al., 2023). At each step an index built from the grammar, typically a finite automaton over the grammar’s states (Willard and Louf, 2023), masks the vocabulary down to the tokens that keep the prefix in the language: a non-conforming continuation is unreachable at decode time, not merely discouraged. XGrammar (Dong et al., 2024) is the engine vLLM exposes for this, and the dialect `tla_module_v1.ebnf` is written in. We are not aware of a published grammar for TLA⁺ itself. `tla_module_v1.ebnf` targets the TLA⁺ expression language specifically, including constructs (nested junction lists, qualified instance application, the symbolic operator set) that a generic programming-language grammar would not cover. We measure it offline; we do not run it inside a generation arm.

TLA⁺ Specification Synthesis. FormaLLM (Bisharat et al., 2026) established the open-loop baseline this program starts from: 26.6% SANY parse and 8.6% TLC model-check across 25 open-weight models. TLA-Prover is the parent training program. TLA⁺ (Lamport, 2002) and TLC (Yu et al., 1999) are the target language and verifier throughout. Both this addendum and the parent program report against industrial TLA⁺ use (Newcombe et al., 2015; Cirstea et al., 2024) as the motivating context, not as a benchmark.

3 Method

Open-Loop Baseline (Framing A). Every prior evaluation in this program, and the control arm here, is open-loop: k independent draws from a single generation prompt, each scored by SANY and then TLC, with the best-of- k result reported per spec. The verifier is a judge, never a controller; a failing draw contributes nothing to the next one. Framing A’s budget is 33 model calls per spec (1 greedy plus $k=32$ best-of- K), or 32 calls per spec in the matched-budget control described in Section 4. Scoring is unchanged

from the parent evaluation: SANY parse, then TLC against the spec’s fixed `.cfg`.

Closed-Loop Framing (Framing L). Framing L replaces the single generation prompt with a chain that reads its own verifier output. Each spec runs 8 independent chains of 4 rounds each, 32 model calls per spec, one fewer than framing A’s 33. The loop cannot win on budget. Chains advance in lock step: all 8 chains complete round r before any chain starts round $r+1$. TLC is strictly serialized across the run; only the network calls to the generation endpoint overlap. Round-0 prompts are byte-identical to framing A’s generation prompt, verified per spec by comparing `prompt_sha256` in the run ledger. Framing L reuses the framing-A scorer verbatim; no scoring code changed between arms.

Diagnosis Rungs. A failing draw is classified into one of four rungs, checked in this priority order: `signature` (the candidate module omits an identifier that the spec’s fixed `.cfg` requires), then `sany` (parse failure), then `tlc_error` (TLC raised an error short of a full run), then `tlc_violation` (TLC ran and found a counterexample). The rung selects which diagnostic text is fed back into the next round’s prompt. The signature check is a selector, not a gate: a signature failure never causes a stage to be skipped, and every candidate still passes through SANY and TLC regardless of what the signature checker reports.

Pre-Registered Analysis Rule. Before any framing-L run was executed, the comparison rule was fixed: a pairing counts as a win only if it reaches at least +5 solved specs over the paired control or McNemar exact $p < 0.05$. Anything smaller than that reports as null, regardless of the sign of the delta. Section 4 applies this rule to all nine (loop seed, open seed) pairings without exception, including the pairings that fail it.

Decode-Time Grammar. `tla_module_v1.ebnf` is a TLA⁺ expression grammar intended for constrained decoding. Its methodology has a gate: a grammar that rejects valid input makes correct output unreachable under constrained decoding, so before any claim about catch rate, the grammar must false-reject zero specs out of a known-good corpus. That known-good corpus is the union of the `tlaplus/examples` corpus and the 30-spec holdout. Only after clearing the zero-false-reject gate is the grammar scored for catch rate: what fraction of real, measured SANY parse failures it rejects. The same gate is applied to a companion `.cfg` grammar against

the corpus’s configuration files. A structural precursor, `tla_module_v0.ebnf`, already existed in the repository before this work and enforces only module framing and bracket balance; it is measured by the same false-reject/catch-rate procedure for comparison. Junction-list alignment, a context-sensitive property that SANY itself parses with a custom lexer rather than a context-free grammar, is out of scope for either grammar version and is not claimed.

4 Results

All numbers in this section come from the base, untuned gpt-oss-120b model against the frozen 30-spec holdout, three seeds per arm unless stated otherwise. Every run reported here is gate-check clean: 0 `api_error` rows, 0 unextracted rows.

Headline Comparison. Framing L solves 18, 16, and 18 of 30 specs across its three seeds (mean 17.3, 571–616 model calls per run). Framing A solves 12, 8, and 12 (mean 10.7, 960–990 model calls per run). The ranges do not overlap: framing L’s worst seed (16) exceeds framing A’s best seed (12). Figure 1 plots solved specs per seed for both arms alongside mean model calls.

Applying the pre-registered rule from Section 3 to a single pairing illustrates why all nine pairings are required rather than one. Loop seed 1 against the same-session open-loop control is +6 specs, $p=0.031$, a win. Loop seed 2 against the same control is +4 specs, $p=0.125$, null by the rule. Reporting seed 1 alone would have been a sampling accident in the favorable direction.

Pairwise Comparison. Table 1 gives all nine (loop seed, open seed) pairings. The delta ranges from +4 to +10 specs. One spec is lost across all nine pairings combined (spec 181, in two of the three pairings against loop seed 3), and no pairing shows a net loss. Sign-test p ranges from 0.0020 to 0.1250 depending on which two seeds are compared.

Per-Spec Stability. Only two specs, 13 and 15, are solved by framing L in every seed and by framing A in none. Specs 132 and 191 are loop-solved in 2 of 3 seeds and never solved open-loop. Specs 32, 106, 133, and 174 are loop-solved in 1 of 3 seeds and never solved open-loop. No spec is ever open-loop-only: framing A never solves a spec that framing L fails on every seed. Figure 2 shows the per-spec solve rate over seeds for both arms. The loop reliably adds specs in the five-to-seven range per seed, but which specs it adds

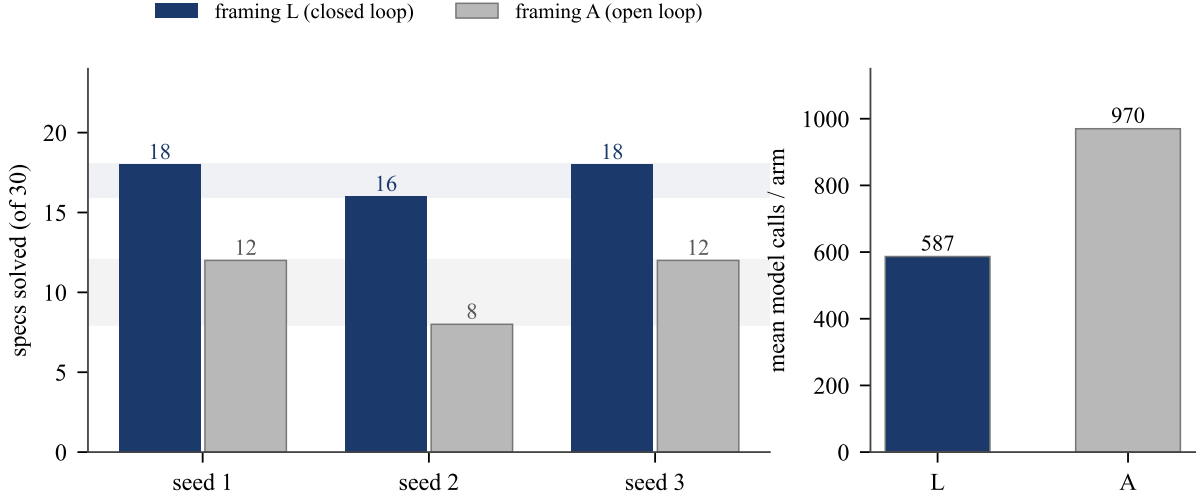


Figure 1: Solved specs per seed for framing L and framing A, with mean model calls per run. The three seeds of each arm are independent runs, not paired with one another or across arms.

Table 1: All nine (loop seed, open seed) pairings, base gpt-oss-120b, 30-spec holdout. Loop seeds solve 18/16/18; open seeds solve 12/8/12. Gained and lost count specs relative to the paired open-loop seed; p is the exact sign-test p -value for that pairing.

L	A	Δ	Gain	Loss	p
1	1	+6	6	0	0.0313
1	2	+10	10	0	0.0020
1	3	+6	6	0	0.0313
2	1	+4	4	0	0.1250
2	2	+8	8	0	0.0078
2	3	+4	4	0	0.1250
3	1	+6	7	1	0.0703
3	2	+10	11	1	0.0063
3	3	+6	6	0	0.0313

varies by seed; the effect is a larger reachable set, not a fixed, deterministic gain.

Mechanism. Loop seed 1 carries per-call proveance for every solved spec. Of its 18 solves, 9 came from a fresh generation in round 0 (the first of the 8 independent draws) and 9 came from a later, repair round. Restricting to the 6 specs gained relative to the frozen open-loop control (13, 15, 106, 174, 181, 191), 5 of the 6 were won on a repair round: three entered the repair prompt at the `sany` rung (specs 13, 106, 174) and two at the `tlc_error` rung (specs 15, 191). Only spec 181 was solved on the first round-0 draw. The advantage over the open-loop control is concentrated in the repair mechanism, not in round-0 sampling diversity.

Confound Checks. Two confounds were closed before reporting the headline numbers. First, endpoint drift: the frozen open-loop control was

measured months before framing L. Re-running the open-loop control on the same warm endpoint at a matched 32-call budget reproduced 12/30 exactly. Per-sample SANY rate across that session, in run order, was 16.6%, 22.9%, 23.9%, 25.7%, 15.2%, with median per-sample latency 20.3s, 23.0s, 22.6s, 22.7s, 20.2s; the third open-loop seed’s 8/30 is spec-level sampling noise on a stable serve, not a degrading endpoint. Second, single-seed accident: three seeds per arm were run and all three are reported for both arms, not a subset chosen after the fact.

Per-sample pass rate is not comparable across the two framings. Framing L stops a spec on its first passing draw, so an easy spec contributes exactly one passing row to the framing-L pool while framing A can bank 20 or more passing rows on the same spec across its $k=32$ draws. Per-sample pass rate is therefore structurally biased against framing L (3.0% vs. 7.5%) and is not a capability comparison between the arms. Per-sample SANY rate carries the same bias in the same direction, which makes framing L’s advantage there conservative rather than inflated: 24.1% (bootstrap 95% CI [22.2, 26.1]) against 14.1% (bootstrap 95% CI [12.8, 15.3]), pooled over 1,760 and 2,910 rows respectively.

Reachability Ceiling. Pooling all six arms (4,670 draws total), 834 draws (18%) pass SANY, 678 (15%) additionally satisfy the `.cfg` signature, and 219 (5%) pass TLC. Figure 3 shows this funnel per draw and per spec. At the spec level, coverage is wider than the per-draw rate suggests: all 30 holdout specs produce at least one parsing

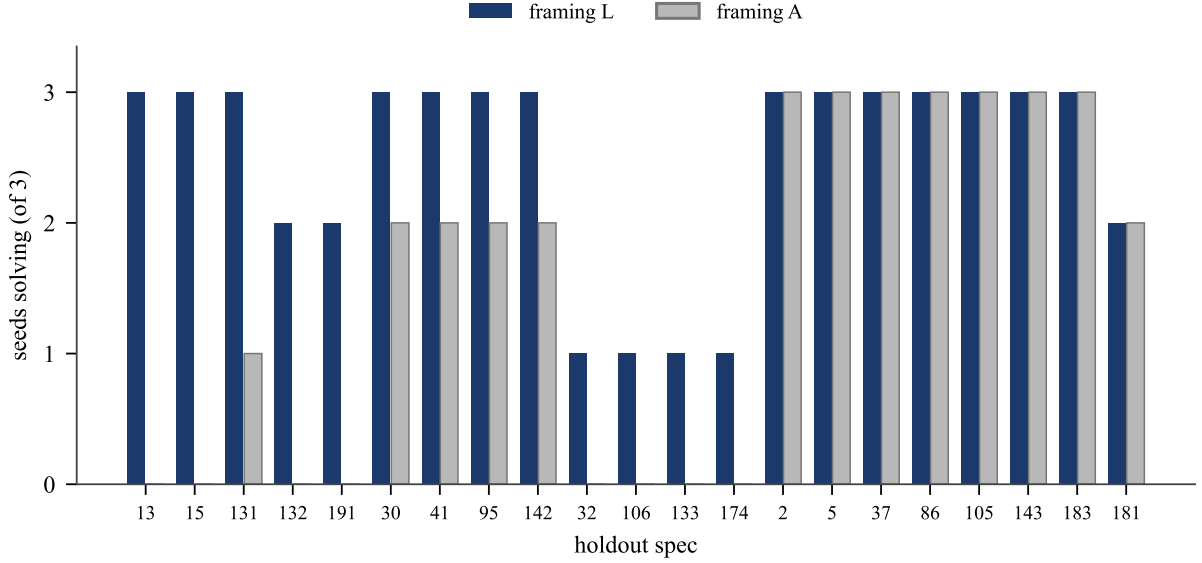


Figure 2: Per-spec solve rate over the three seeds of each arm, framing L against framing A.

candidate somewhere across the six arms, 26 of 30 produce at least one signature-complete candidate, and 20 of 30 produce at least one TLC pass. The worst spec by density is 106, with 1 passing draw out of 174; the best is 181, with 101 out of 158. Spec-level SANY coverage is already 100%: the ceiling on solvable specs is set by TLC and signature completion, not by parsing.

Failure Taxonomy. 3,836 SANY failures were classified across the six arms. Grammar failures (the module does not parse) account for 1,967 (51%); unknown-operator errors account for 1,304 (34%); the remainder splits across an unclassified other category (375, 10%), duplicate definitions (97, 3%), wrong arity (86, 2%), and missing substitution or level errors (7, 0%). Figure 4 breaks down both the top-level classes and the unknown-operator subclasses.

Of 3,750 unknown-operator errors located in the candidate’s own module, 1,990 (53%) reference an operator that exists nowhere in the module or its imports, 1,195 (32%) qualify a name against an unqualified `INSTANCE`, 385 (10%) are a bound variable escaping its quantifier’s scope, 97 (3%) use `RECURSIVE` without a preceding declaration, and 83 (2%) are a genuine forward reference. The `INSTANCE` case is one misunderstanding that produces two error classes at once: the model writes an unqualified `INSTANCE Bakery`, which imports every name from `Bakery` directly, and then separately writes `vars == Bakery!vars`, a qualified reference that requires a named instance; the redefinition then collides with the

name the unqualified `INSTANCE` already imported. The five most frequent offending tokens in the parse-failure population are : (139), `\subsetq` (126), , (118), `AS` (89), and `LET` (85), corresponding respectively to comma-separated `LET` bindings, `\E x \subsetq S, INSTANCE M AS B`, lowercase `let...in`, and `\Sum i \in S :.` Each is a construct borrowed from a more common programming language.

Lint Negative Result. A deterministic lint pass (drop declarations that collide with an `EXTENDS`, add a missing standard-module `EXTENDS`, drop duplicate definitions) was applied to the unsolved specs. It recovered 88 candidates across 9 specs from fail-to-parse to parsing. Specs flipped from unsolved to solved by this pass: zero. All 88 recovered candidates subsequently failed at TLC. Forcing output to parse, on its own, does not translate into a solved spec.

Fine-Tuning Comparison. The best fine-tuned checkpoint measured in this program reaches 15/30 open-loop against a base-model control of 12/30: +3 specs, 5 gained, 2 lost, McNemar exact $p=0.45$, null by the pre-registered rule. The untuned base model inside the loop (mean 17.3/30) exceeds this fine-tuned checkpoint’s open-loop score. The 15/30 also decomposes further: 4 of those specs are library modules scored on SANY alone, 1 is a proof module scored on TLAPS, leaving 10 specs actually model-checked by TLC.

Grammar Measurement. tla_module_v1.

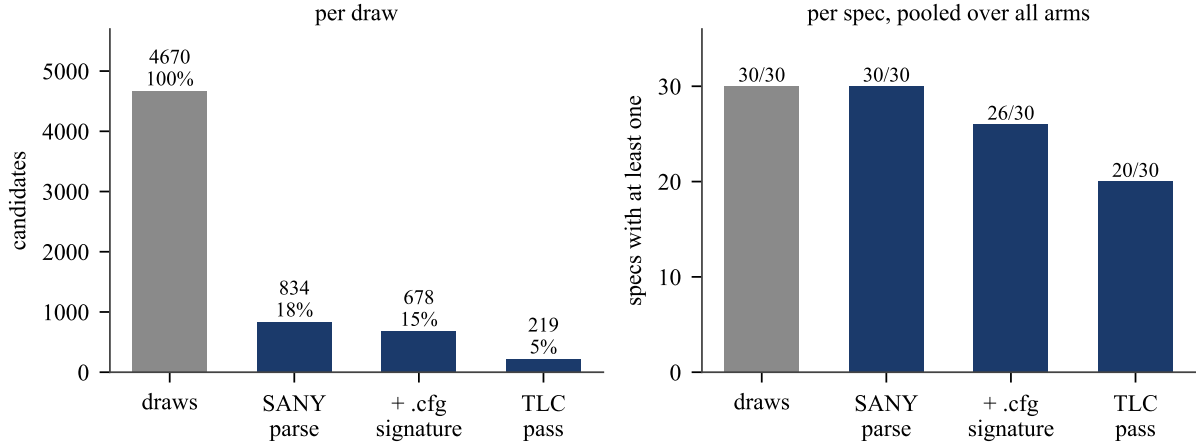


Figure 3: Draws, SANY parse, .cfg signature completion, and TLC pass, pooled over all six arms and broken out per draw and per spec.

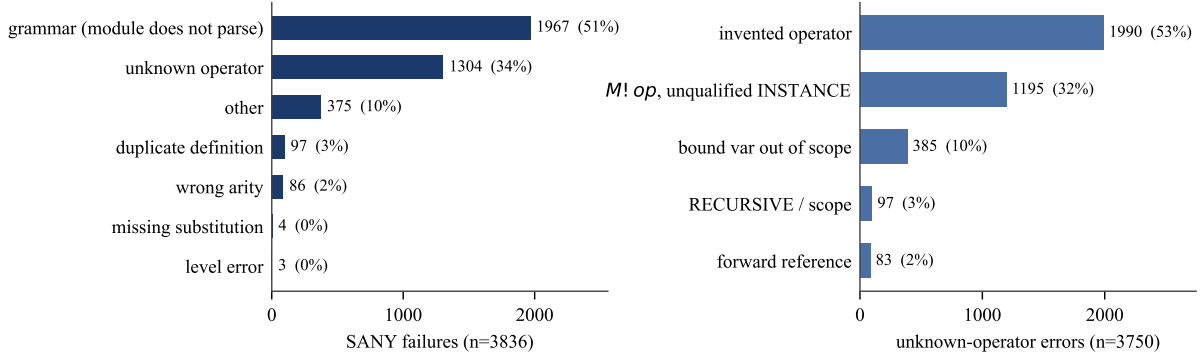


Figure 4: SANY failure classes across the six arms, and the breakdown of the unknown-operator class into its five subtypes.

ebnf was scored against 438 known-good specs (the examples corpus plus the 30-spec holdout) and 1,543 measured real parse failures. It clears the false-reject gate: 0 of the 438 known-good specs are rejected. Against real failures it rejects 1,338 of 1,543 (86.7%). The structural precursor already in the repository, `tla_module_v0.ebnf`, false-rejects 5.0% of the known-good specs and catches 3.2% of real parse failures; because the candidates are already 100% well-framed, balanced, and terminated, the structural tier has almost nothing left to act on. A companion .cfg grammar false-rejects 0 of 230 corpus configuration files. Figure 5 compares the two module-grammar versions on false-reject rate and catch rate.

The constructs that forced grammar iteration, none of them guessable from a language reference alone, include nested junction lists under a single bullet, bare `\` as set difference, postfix application `f[x]`, `\leq` and `\geq`, operator symbols passed as arguments (`Fold(+, 0, ...)`), tuple subscripts

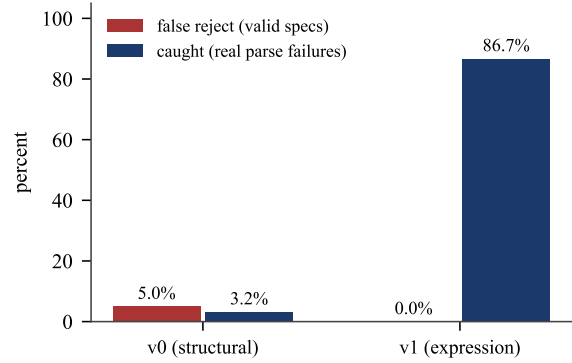


Figure 5: Grammar v0 against v1: false-reject rate on the 438-spec known-good corpus and catch rate on 1,543 measured parse failures.

in `[] [Next] _<<a,b>>`, qualification after an application (`P(Succ)!Reachable0`), the parenthesised infix operators `(+)`, `(-)`, `(\X)`, the symbolic operator set `^ ^ ** ++ - // % # $ $ ? ?`, `!!`, `RECURSIVE` declared inside a `LET`, nested `--` `MODULE Inner` `--` `units`, nesting block comments

(`( * * ) *`), subexpression labels `P0 :: expr`, and identifiers beginning with a digit (`-- MODULE 2PCwithBTM --` is a real corpus spec). Junction-list alignment is not enforced by either grammar version and is not addressable by a context-free grammar: it is context-sensitive, and SANY itself uses a custom lexer for it rather than its context-free rules.

5 Limitations

The grammar has not been run inside a generation arm. The 86.7% catch rate is an offline measurement over saved candidates, an upper bound on what decode-time constraint would remove. No claim about pass rate under constrained decoding is supported by this addendum.

The 2x2 is incomplete. Framing L has been measured on the base model only. The tuned-model cell requires a self-hosted serve that has not yet run, so this addendum does not report what task training adds on top of the loop.

Every result in this addendum comes from one model family, one model size (gpt-oss-120b), and one 30-spec holdout.

Spec-level pass@k at $k=32$ is noisy: the open-loop control moved 12, 8, and 12 across its three seeds while its per-sample SANY rate stayed flat.

Constrained decoding is requested under a parameter name that changed. vLLM 0.22 renamed `guided_grammar` and `guided_choice` to `structured_outputs`; the legacy names are accepted with HTTP 200 and silently ignored, with no error and no warning. Sending `guided_choice: ["alpha","beta"]` returns a free-form sentence, while `structured_outputs: {"choice": [...]}` returns `alpha`. Both endpoints used here enforce the current form. An arm that sends the legacy name is ungated and reports as though it were gated, so we check enforcement with two requests before any grammar-gated run.

4 of the 30 holdout specs are library modules scored on SANY alone and 2 are proof modules scored on TLAPS; 24 of the 30 are graded by TLC.

6 Conclusion

Every evaluation in TLA-Prover, including its own headline number, treats the verifier as a

judge: score k draws, report the best. Feeding the verifier’s diagnosis back into regeneration, on an untuned base gpt-oss-120b and at a lower model-call budget than the open-loop control, raises the mean solved count on the frozen 30-spec holdout from 10.7/30 to 17.3/30 across three seeds each, with non-overlapping ranges. It also exceeds 15/30, the best open-loop result this training program has produced with fine-tuning, a result that the program’s own pre-registered rule already marks null. Half of framing L’s solves in the seed with per-call provenance come from a fresh draw and half from a repair round; five of the six specs gained over the open-loop control were won on repair, not on generation. Only two of the 30 holdout specs are solved by every closed-loop seed and no open-loop seed; the rest of the gain comes from a larger reachable set whose composition shifts by seed, not from a fixed set of specs the loop alone can reach.

The failure taxonomy and the decode-time grammar bound what closing the loop is working against and what remains unaddressed. 51% of pooled SANY failures do not parse, and a grammar built for the TLA⁺ expression language rejects 86.7% of 1,543 real parse failures with zero false rejects on 438 known-good specs. That number describes what a decode-time constraint would remove from the failure population, not a measured pass-rate gain: the grammar has not been run inside a generation arm, and a companion lint pass that forces 88 failing candidates to parse converts zero of them to a TLC pass. Forcing a candidate past the parser does not by itself get it past the model checker. The 2x2 this addendum motivates, framing (open vs. closed) by training (base vs. fine-tuned), has one cell unmeasured: framing L has run on the base model only, and the tuned-model cell requires a self-hosted serve that has not yet run, so nothing here says what task training adds on top of the loop. A related but separate gap is decode-time constraint itself: the shared inference endpoint used for every arm in this addendum accepts vLLM’s `guided_grammar` and `guided_choice` parameters with HTTP 200 and applies neither, so the grammar’s 86.7% catch rate cannot yet be turned into a generation-arm result on this endpoint. Closing either gap requires a self-hosted serve, which has not yet run.

ACKNOWLEDGEMENTS

This work used resources of the Argonne Leadership Computing Facility.

(CHARME), volume 1703 of *Lecture Notes in Computer Science*, pages 54–66. Springer.

REFERENCES

- Bisharat, A., Laufer, K., Spencer, E., Thiruvathukal, G. K., et al. (2026). FormaLLM: A systematic evaluation of large language models for TLA+ specification generation from natural language. In *Proceedings of the International Conference on Software Technologies (ICSOFT)*.
- Cirstea, H., Kuppe, M. A., Loillier, B., and Merz, S. (2024). Validating traces of distributed programs against TLA⁺ specifications. In *Software Engineering and Formal Methods (SEFM 2024)*, volume 15280 of *Lecture Notes in Computer Science*, pages 126–143. Springer.
- Dong, Y., Ruan, C. F., Cai, Y., Lai, R., Xu, Z., Zhao, Y., and Chen, T. (2024). XGrammar: Flexible and efficient structured generation engine for large language models. *arXiv preprint arXiv:2411.15100*.
- First, E., Rabe, M. N., Ringer, T., and Brun, Y. (2023). Baldur: Whole-proof generation and repair with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1229–1241.
- Geng, S., Josifoski, M., Peyrard, M., and West, R. (2023). Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, Singapore.
- Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley Longman Publishing Co., Inc.
- Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., and Deardeuff, M. (2015). How Amazon web services uses formal methods. *Communications of the ACM*, 58(4):66–73.
- OpenAI (2025). gpt-oss-120b & gpt-oss-20b model card. <https://huggingface.co/openai/gpt-oss-20b>.
- Willard, B. T. and Louf, R. (2023). Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.
- Xin, H., Guo, D., Shao, Z., Ren, Z., Zhu, Q., Liu, B., Ruan, C., Li, W., and Liang, X. (2024). DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data. *arXiv preprint arXiv:2405.14333*.
- Yu, Y., Manolios, P., and Lamport, L. (1999). Model checking TLA+ specifications. In *Correct Hardware Design and Verification Methods*